Lecture 16 — Graphs I: Representations, BFS, DFS, Components, Grids, Bipartite Check

> Year Curriculum · Prerequisites: Week 2 (STL: `vector`, `queue`, `stack`), Week 8 (recursion) · Estimated time: 2.5 hours

1. Overview

A graph is the most versatile model in competitive programming. Roads between cities, friendships, cells of a maze, positions of a puzzle, even the states of a dynamic program — all of them are vertices joined by edges. This lecture covers the foundation every later graph topic is built on:

- how to **store** a graph efficiently,
- **breadth-first search (BFS)**, which computes shortest paths when every edge costs the same,
- **depth-first search (DFS)**, which explores structure,
- **connected components**,
- **grids as implicit graphs**, and
- testing whether a graph is **bipartite**.

Every algorithm today runs in O(n + m) time, where n is the number of vertices and m the number of edges. That is linear in the input size, so graphs with hundreds of thousands of vertices and edges are routine.

2. Vocabulary and representations

2.1 Terms

A graph G = (V, E) has a set of vertices V (|V| = n) and edges E (|E| = m). An edge may be **undirected** ({u, v}: travel both ways) or **directed** (u → v: one way only). A graph may be **weighted** (each edge has a length or cost; Week 17). The **degree** of a vertex is its number of incident edges; in an undirected graph the degrees sum to 2m (each edge is counted at both ends).

A **path** is a sequence of vertices with consecutive ones adjacent; a **cycle** is a path that returns to its start. A graph is **connected** if every pair of vertices is joined by a path; a **connected component** is a maximal connected piece. A **tree** is a connected graph with no cycles; it has exactly $n - 1$ edges.

Contest graphs may contain **self-loops** (u–u) and **multi-edges** (several u–v edges) unless the statement promises otherwise. Algorithms in this lecture handle both correctly, but code that, say, counts neighbours to compute something may not.

2.2 Three representations

Adjacency matrix. `bool a[n][n]`, `a[u][v] = true` if u–v is an edge. Edge lookup is O(1), but memory is $O(n^2)$ and iterating over the neighbours of u costs O(n) even if u has one neighbour. Use it only for n up to a few thousand, or when an algorithm naturally works on a matrix (Floyd-Warshall, Week 17).

Adjacency list. `vector<vector<int>> adj(n)`, where `adj[u]` lists the neighbours of u. Memory O(n + m), iterating over neighbours costs O(deg u). This is the default for almost every problem.

Edge list. `vector<array<int,3>> edges` holding (u, v, w). Useful when the algorithm processes edges globally rather than by vertex: Kruskal's MST (Week 19), Bellman-Ford (Week 17).

For $n = 10^5$ an adjacency matrix would need 10^{10} entries; the list needs a few megabytes. Reading an undirected graph into a list:

```
vector<vector<int>> adj(n + 1);   // vertices 1..n
for (int i = 0; i < m; i++) {
    int a, b; cin >> a >> b;
    adj[a].push_back(b);
    adj[b].push_back(a);          // only for undirected graphs
}
```

Before writing this, check in the statement: are vertices 0- or 1-indexed? Is the graph directed? Are there weights?

3. Breadth-first search

3.1 The algorithm

BFS explores the graph in layers. Starting from a source s, it first visits s (distance 0), then all neighbours of s (distance 1), then all their unvisited neighbours (distance 2), and so on. A FIFO queue implements this naturally:

```cpp
vector<int> dist(n + 1, -1);
queue<int> q;
dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u])
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
}
```

`dist[v] == -1` doubles as the "not yet visited" flag.

3.2 Why BFS finds shortest paths

Claim. At every moment, the queue contains vertices whose distances are d, d, …, d, d+1, …, d+1 for some d (non-decreasing, spanning at most two consecutive values), and each vertex's `dist` equals its true shortest distance from s.

Sketch: initially the queue is just s with distance 0. Popping a vertex u with `dist = d` pushes neighbours with `dist = d+1` at the back, preserving the property. Now suppose vertex v has true distance D and let u be its predecessor on a shortest path (true distance D−1). By induction u is assigned D−1, and v is discovered no later than when u is popped; it cannot be discovered earlier from a vertex at distance < D−1, since then v's true distance would be < D. So v gets exactly D.

The consequence: in a graph where every edge has the same cost, **BFS computes single-source shortest paths in O(n + m)**. With weights you need Dijkstra (Week 17); with weights 0 and 1 only, 0-1 BFS (Week 17).

3.3 Mark when pushing

Set `dist[v]` (or `visited[v]`) at the moment v is *pushed*, not when it is popped. If you mark on pop, a vertex can be pushed many times by different neighbours before it is popped; on dense graphs the queue grows to O(m) and, in grids, the program can time out or produce wrong distances.

3.4 Reconstructing the path

Store `par[v] = u` when v is discovered from u. The parent pointers form the **BFS tree**; walking from the target back to the source and reversing gives a shortest path. CSES 1667 *Message Route* asks for exactly this — see §8.1.

3.5 Multi-source BFS

To compute, for every vertex, the distance to the *nearest* of several sources, push all sources with distance 0 before starting. The layer argument is unchanged. This is how CSES 1194 *Monsters* computes when each cell becomes dangerous, and how "distance to the nearest hospital" style problems are solved in one pass instead of one BFS per source.

4. Depth-first search

4.1 The algorithm

DFS follows one path as deep as possible, then backtracks:

```
vector<bool> vis(n + 1, false);
void dfs(int u) {
    vis[u] = true;
    for (int v : adj[u])
        if (!vis[v]) dfs(v);
}
```

It also runs in O(n + m): each vertex is entered once and each adjacency list scanned once. DFS does **not** compute shortest paths — on a cycle 1–2–3–4–1 starting at 1 it may reach 4 via 2 and 3 although 4 is adjacent to 1.

What DFS gives instead is *structure*. The recursion naturally knows when a vertex is entered and when it is finished, which we will use for cycle detection and topological sort (Week 21), subtree sizes and tree DP (Week 20), and Euler tours (Week 22).

4.2 Recursion depth

On a path of 10^5 vertices, recursive DFS nests 10^5 calls. Each call uses some stack memory, and the default stack is 8 MB on Linux and only 1 MB on Windows. Codeforces sets a large stack, but other judges may not, and your local test may crash where the judge would not (or vice versa). On a 1000 × 1000 grid the depth can reach 10^6 — that is risky anywhere.

Two safe options:

1. Use BFS when you only need reachability or components.
2. Write DFS with an explicit stack:

```
vector<int> st = {s};
vis[s] = true;
while (!st.empty()) {
    int u = st.back(); st.pop_back();
    for (int v : adj[u])
```

```
        if (!vis[v]) { vis[v] = true; st.push_back(v); }
}
```

This visits exactly the same set of vertices but not in the same order as recursive DFS, and it does not have "finish" events. When you need finishing order (Week 21), the iterative version must store an iterator index per stack frame.

5. Connected components

To find all components, start a search from every vertex that is still unvisited:

```
int comps = 0;
for (int v = 1; v <= n; v++)
    if (!vis[v]) { comps++; dfs(v); }
```

Although there may be many searches, each vertex and each edge is processed in exactly one of them, so the total is still O(n + m). Storing `comp[v] = id` instead of a boolean lets you answer "are u and v in the same component?" in O(1) afterwards.

Worked example: CSES 1666 *Building Roads*. Given n cities and m roads, add the minimum number of roads to make everything connected and print them. If there are k components, k − 1 roads are necessary (each new road reduces the number of components by at most one) and sufficient: pick one representative from each component and join representative i to representative i + 1. See §8.2.

When edges are *added* over time and you need connectivity after each addition, re-running DFS is too slow; the Disjoint Set Union structure (Week 19) handles that in near-constant time per operation.

6. Grids as implicit graphs

A grid of characters is a graph whose vertices are cells and whose edges join adjacent open cells. We never build an adjacency list; neighbours are generated on the fly with direction arrays:

```
const int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
for (int d = 0; d < 4; d++) {
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;   // bounds BEFORE indexing
    if (g[nr][nc] == '#' || seen[nr][nc]) continue;
    // visit (nr, nc)
}
```

Changing the arrays gives 8-neighbourhoods or knight moves. A grid with R rows and C columns has R·C vertices and at most 4·R·C directed adjacencies, so BFS is O(R·C).

Flood fill — CSES 1192 *Counting Rooms*. Count the connected regions of `.` cells. Scan the grid; for every unseen floor cell, increase the counter and BFS from it marking the whole room. Example:

```
########
#..#...#
####.#.#
#..#...#
########
```

The rooms are: the two cells at the top left; the ring on the right (cells around the `#` at row 2, column 5 are all connected); and the two cells at the bottom left — 3 rooms.

Shortest path in a maze — CSES 1193 *Labyrinth*. BFS from `A`. To print the path as a string of `U/D/L/R`, store for each cell the direction letter used to enter it; walk backwards from `B` by undoing those moves, then reverse the string.

7. Bipartite graphs

A graph is **bipartite** if its vertices can be split into two sets so that every edge goes between the sets — equivalently, if it can be properly coloured with two colours. A classic theorem: *a graph is bipartite if and only if it contains no cycle of odd length.*

The algorithm follows directly. Run BFS (or DFS) from an uncoloured vertex, give it colour 1, and give every newly discovered vertex the colour opposite to its discoverer. If we ever find an edge whose endpoints already have the same colour, the graph is not bipartite. Why is that correct? In a single component, the colour of a vertex is forced by the colour of the start (each step flips it), so if the forced colouring fails, no colouring works. And if the check passes for every edge, we have exhibited a valid colouring.

Remember to restart from every uncoloured vertex: each component can be coloured independently. CSES 1668 *Building Teams* is exactly this problem (§8.4).

Bipartite structure shows up disguised: "split the students into two teams so that no two friends are in the same team", "can this be a chessboard-like pattern", "assign each node to day or night shift". Bipartite graphs will also be central for matching in Week 32.

8. Reference implementations

All compile with `g++ -std=c++17 -O2 -Wall` and produce the expected sample output.

8.1 BFS with path reconstruction — CSES 1667 *Message Route*

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vector<int> dist(n + 1, -1), par(n + 1, 0);
    queue<int> q;
    dist[1] = 0;
    q.push(1);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u])
            if (dist[v] == -1) {          // first time seen = shortest distance
                dist[v] = dist[u] + 1;
                par[v] = u;
                q.push(v);
            }
    }
    if (dist[n] == -1) { cout << "IMPOSSIBLE\n"; return 0; }
    vector<int> path;
    for (int v = n; v != 1; v = par[v]) path.push_back(v);
    path.push_back(1);
    reverse(path.begin(), path.end());
    cout << path.size() << "\n";
    for (int v : path) cout << v << " ";
    cout << "\n";
}
```

Sample (5 vertices, edges 1-2, 1-3, 1-4, 2-3, 5-4) → `3` and `1 4 5`.

8.2 Components with iterative DFS — CSES 1666 *Building Roads*

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, m;
vector<vector<int>> adj;
vector<int> comp;
void dfs(int start, int id) {             // iterative DFS: no recursion-depth issues
    vector<int> st = {start};
    comp[start] = id;
    while (!st.empty()) {
```

```
        int u = st.back(); st.pop_back();
        for (int v : adj[u])
            if (comp[v] == -1) { comp[v] = id; st.push_back(v); }
    }
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    adj.assign(n + 1, {});
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    comp.assign(n + 1, -1);
    vector<int> leaders;                    // one vertex per component
    for (int v = 1; v <= n; v++)
        if (comp[v] == -1) { dfs(v, leaders.size()); leaders.push_back(v); }
    cout << leaders.size() - 1 << "\n";
    for (size_t i = 1; i < leaders.size(); i++)
        cout << leaders[i - 1] << " " << leaders[i] << "\n";
}
```

8.3 Grid flood fill — CSES 1192 *Counting Rooms*

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &row : g) cin >> row;
    const int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
    vector<vector<bool>> seen(n, vector<bool>(m, false));
    int rooms = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (g[i][j] != '.' || seen[i][j]) continue;
            rooms++;
            queue<pair<int, int>> q;        // flood fill this room with BFS
            q.push({i, j});
            seen[i][j] = true;
            while (!q.empty()) {
                auto [r, c] = q.front(); q.pop();
                for (int d = 0; d < 4; d++) {
                    int nr = r + dr[d], nc = c + dc[d];
                    if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
                    if (g[nr][nc] != '.' || seen[nr][nc]) continue;
                    seen[nr][nc] = true;
```

```
                    q.push({nr, nc});
                }
            }
        }
    cout << rooms << "\n";
}
```

8.4 Bipartite check — CSES 1668 *Building Teams*

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vector<int> color(n + 1, 0);           // 0 = uncoloured, 1 / 2 = teams
    for (int s = 1; s <= n; s++) {
        if (color[s]) continue;            // graph may be disconnected
        color[s] = 1;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == 0) { color[v] = 3 - color[u]; q.push(v); }
                else if (color[v] == color[u]) { cout << "IMPOSSIBLE\n"; return 0; }
            }
        }
    }
    for (int v = 1; v <= n; v++) cout << color[v] << " ";
    cout << "\n";
}
```

Sample (edges 1-2, 1-3, 4-5) → `1 2 2 1 2`; a triangle → `IMPOSSIBLE`.

9. Common mistakes

- **Marking visited on pop** instead of on push: duplicates in the queue, TLE or wrong distances.
- **Missing reverse edge** in undirected graphs (or adding one in directed graphs).
- **Disconnected graphs**: searching only from vertex 1 for components or bipartiteness.

- **Grid bounds**: check `0 <= nr < n` before reading `g[nr][nc]`. Negative indices on `vector` are undefined behaviour, not an exception.
- **Stack overflow** in recursive DFS on long paths or large grids.
- **Index base mismatch**: `adj(n)` with vertices numbered 1..n.
- **Reusing arrays across test cases** without clearing (`adj`, `vis`, `dist`). Clear exactly the part you used — `adj[i].clear()` for i in 0..n — not a global array of size 10^6 per test.
- **Using BFS on weighted graphs**: BFS minimises the number of edges, not the total weight.

10. Practice problems

- CSES 1192 — Counting Rooms — https://cses.fi/problemset/task/1192
- CSES 1193 — Labyrinth — https://cses.fi/problemset/task/1193
- CSES 1666 — Building Roads — https://cses.fi/problemset/task/1666
- CSES 1667 — Message Route — https://cses.fi/problemset/task/1667
- CSES 1668 — Building Teams — https://cses.fi/problemset/task/1668
- CSES 1669 — Round Trip (find any cycle with DFS parents) — https://cses.fi/problemset/task/1669
- CSES 1194 — Monsters (multi-source BFS) — https://cses.fi/problemset/task/1194
- Codeforces 862B — Mahmoud and Ehab and the bipartiteness — https://codeforces.com/problemset/problem/862/B
- Codeforces 1037D — Valid BFS? — https://codeforces.com/problemset/problem/1037/D
- Codeforces 377A — Maze (DFS on a grid) — https://codeforces.com/problemset/problem/377/A
- USACO 2016 December Silver — Moocast — http://www.usaco.org/index.php?page=viewproblem2&cpid=668

11. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapters 11 (Basics of graphs) and 12 (Graph traversal).
- CP-Algorithms: "Breadth-first search" (https://cp-algorithms.com/graph/breadth-first-search.html) and "Depth First Search" (https://cp-algorithms.com/graph/depth-first-search.html).
- USACO Guide, Silver: "Graph Traversal" and "Flood Fill" modules (https://usaco.guide/silver/).
- Looking ahead: Week 17 extends BFS to weighted shortest paths; Week 19 handles dynamic connectivity with DSU; Week 21 uses DFS finishing times for topological sort and SCCs.